Monte-Carlo Pricing for American Style Options and Comparisons to the Market

Kasper Kujala, James Schimmel,
Aryaman Utkarsh Gangwar, and Adam Tyler

Introduction

Accurately pricing financial instruments is critical to maintaining a solid trading strategy and preventing arbitrage through mismatched prices. We use the Monte Carlo simulation method to evaluate various options that are available on the market today. We aim to see how closely the Monte-Carlo simulation agrees with the market prices and potentially inform us about arbitrage opportunities. Compared to market prices, our simulations follow the actual market price very closely; during high volatility, the simulated price can become inaccurate as the constant volatility characteristic differs from the absolute fluctuations.

Literature Review-

Lind and Poulsen(2024:4) introduced the Least Squares Monte Carlo Simulation, an enhancement of the classical Least Squares Approach. The method mainly aimed to improve the estimation of the coefficients, thus getting more accurate pricing of American Options by including both discounted payoff and its derivative. Djagba and Ndizihiwe(2024:3) explored the application of machine learning algorithms for pricing American Options through Monte Carlo simulation. The research was focused on addressing the limitations of traditional models used like BSM in terms of handling early exercise features and the non-linear payoff nature of American Options. They proposed using Monte Carlo and Machine learning to improve pricing accuracy and efficiency. Zhu and Stentoft(2024:2) examined the Weighted Least Squares Monte Carlo, focusing on correcting heteroscedasticity in regression-based continuation value estimation. The findings of this research showed that the estimators(corrected) were accurate away from the

exercise price; however, they did not significantly improve exercise decisions or option price estimates. Huo (2023:1) presented a method for enhancing the accuracy and stability of Least-Sqaures Monte Carlo for American-style option pricing. He constructed an ansatz of conditional expect continuation payoff, which used exact solutions from low-dimensional finite difference methods, therefore bridging the gap between solving back partial differential equations and Monte Carlo Simulation. Molent and Zannette (2019:6) proposed an efficient method for pricing multi-asset American options, combining Monte Carlo with machine learning and Variance reduction techniques. The author used European option price as a control variate, which enables the handling of high-dimensional baskets while reducing the variance for price estimators. Glau Mahlstedt and Pötz (2018:5) introduced pricing American Options using the Dynamic Chebyshev method. Chebyshev approximated the value function polynomials during the dynamic programming time-stepping process. This method allowed for shifting model-dependent computations into an offline phase.

Least Squares Monte Carlo simulation

Least Squares Monte Carlo (LSMC) simulation is a numerical method used to value options, particularly those with complex features like early exercise, which are commonly found in American or Bermudan options. The approach combines the power of Monte Carlo simulation to model the stochastic paths of underlying asset prices shown in formula [1].

$$S_t^{(i)} = \mathrm{F}(S_{t-1}^{(i)}, \varepsilon_t^{(i)}, \Delta t) \tag{1}$$

S_t is the underlying asset for t= 0, 1, 2, …, T. Then the payoff at maturity T is calculated with formula [2].

$$V_T^{(i)} = Max(0,\ payoff(S_T^{(i)})) \tag{2}$$

The continuation value of the option is shown in [3].

$$C_t^{(i)} = E[V_{t+1} e^{-r\Delta t} \mid S_t^{(i)}] \tag{3}$$

We use polynomial regression techniques to estimate the continuation value of the option at each time step. In essence, it simulates a large number of possible price paths for the underlying asset, then works backward in time, comparing the immediate exercise value to the expected future value of holding the option. If the immediate exercise value exceeds the continuation value, the option is exercised; otherwise, it is held. This comparison is shown in [4].

$$V_t^{(i)} = \begin{cases} payoff\ (S_t^{(i)}), & if\ payoff(S_t^{(i)}) > C_t^{(i)} \\ V_{t+1}^{(i)} e^{-r\Delta t}, & otherwise \end{cases} \tag{4}$$

The continuation value is estimated using least squares regression, which fits a polynomial function [5] to the simulated data.

$$C_t \approx \beta 0 + \beta_1 S_t^{(i)} + \beta_2 (S_t^{(i)})^2 + ... \quad (5)$$

This method is particularly useful for options with path-dependent features, complex payoff structures, or options that allow for early exercise, as it avoids the need for nested simulations and can handle high-dimensional problems efficiently. By using LSMC, we can obtain accurate estimates of option prices even in scenarios where traditional methods (such as finite difference methods or binomial trees) are difficult to apply. Using the value of the option at each iteration we can then find the value of the option at time zero [6]. Where N is the number of Monte Carlo Simulations we perform.

$$V_o = e^{-rT} \frac{1}{N} \sum_{i=1}^{N} V_0^{(i)} \quad (6)$$

Polynomial Regression

Polynomial regression, specifically a quadratic regression, is used to estimate the continuation value of the American option. Which represents the expected payoff of holding the option rather than exercising it at a particular time step.

To approximate the continuation value, we use the following basis functions:

$\Phi_1(S_j) = 1, \Phi_2(S_j) = S_j, \Phi_3(S_j) = S_j^2$

Where S_j is the asset price at time *j*.

These basis functions allow us to model both linear and non linear dependencies of the continuation value on the asset price.

The target variable for the regression is the discounted cash flows from the next time step.

$$Y_i = e^{-r\Delta t} + V_{j+1,i}(S_{j+1,i})$$

Where:

- r is the risk-free rate,
- Δt is the time increment,
- $V_{j+1,i}(S_{j+1,i})$ represents the value function at the next time step (j+1) for path i, obtained from either intrinsic value or continuation value.

The value function representing the intrinsic value of the options is give by:

$$V_j(S_{j,i}) = max(K - S_{j,i}, 0)$$

Where:

- K is the strike price of the option,
- $S_{j,i}$ is the asset price at time j for the i-th path.

The regression model used to approximate the continuation value is:

$$\alpha_{1,M-1} + \alpha_{2,M-1}S_{M-1,i} + \alpha_{3,M-1}S^2_{M-1,i} + \epsilon_i = e^{-r\Delta t}max(K - S_{M,i}, 0),$$

$$i \in I_{M-1}$$

Where:

- I_{M-1}: The set of paths where the option is in the money at time M-1.
- $S_{M-1,i}$: The asset price at time M-1 for the i-th path.
- $S_{M,i}$: The asset price at time step M, for the i-th path.

- $e^{-r\Delta t} max(K - S_{M,i}, 0)$: The discounted payoff at time M.
- $\alpha_{1,M-1}, \alpha_{2,M-1}, \alpha_{3,M-1}$: The regression coefficients corresponding to the constant term, linear term, and quadratic term.
- ϵ_i: The residuals for each data point

The regression coefficients are the set weights which allow us to to approximate the continuation value. The weights are linked to the time period and not the sample paths.

The resulting continuation value approximation is:

$$\hat{C}(S_{M-1}) = \alpha_{1,M-1} + \alpha_{2,M-1} S_{M-1} + \alpha_{3,M-1} S^2_{M-1}$$

Data

For our reference data, we used Theta Data. They are a financial data platform tailored for quantitative researchers, providing access to high-quality options and market data. The data we utilized consisted of 1-minute interval underlying asset prices and corresponding option value data, spanning the period from November 4th 2024 to the expiration date on November 8th 2024. The following Data Frames show what the data looks like for the underlying price and the option we plan to price for AMD.

Option Data:

	bid_size	bid_exchange	bid	bid_condition	ask_size	ask_exchange	ask	ask_condition	date	time
0	0	42	0.00	50	0	42	0.00	50	2024-11-04	2024-11-04 09:30:00
1	12	73	1.82	50	80	69	1.85	50	2024-11-04	2024-11-04 09:31:00
2	149	22	1.70	50	32	76	1.74	50	2024-11-04	2024-11-04 09:32:00
3	52	7	1.63	50	5	65	1.67	50	2024-11-04	2024-11-04 09:33:00
4	40	43	1.67	50	28	46	1.71	50	2024-11-04	2024-11-04 09:34:00
...	...	...	...	...	...	...	...	...	...	...
1950	33	11	2.87	50	58	11	3.00	50	2024-11-08	2024-11-08 15:56:00
1951	9	11	2.86	50	22	11	2.92	50	2024-11-08	2024-11-08 15:57:00
1952	60	9	2.94	50	65	11	3.10	50	2024-11-08	2024-11-08 15:58:00
1953	31	11	2.98	50	59	11	3.10	50	2024-11-08	2024-11-08 15:59:00
1954	23	11	2.91	50	24	11	3.10	50	2024-11-08	2024-11-08 16:00:00

Underlying Data:

	bid_size	bid_exchange	bid	bid_condition	ask_size	ask_exchange	ask	ask_condition	date	time
0	6	7	141.58	0	1	7	141.70	0	2024-11-04	2024-11-04 09:30:00
1	6	65	140.51	0	1	1	140.53	0	2024-11-04	2024-11-04 09:31:00
2	8	75	140.12	0	2	1	140.20	0	2024-11-04	2024-11-04 09:32:00
3	1	1	139.95	0	8	75	139.99	0	2024-11-04	2024-11-04 09:33:00
4	5	65	140.16	0	1	1	140.22	0	2024-11-04	2024-11-04 09:34:00
...	...	...	...	...	...	...	...	...	...	...
1950	2	68	147.89	0	1	7	147.93	0	2024-11-08	2024-11-08 15:56:00
1951	15	75	147.86	0	4	68	147.89	0	2024-11-08	2024-11-08 15:57:00
1952	10	1	147.99	0	25	1	148.00	0	2024-11-08	2024-11-08 15:58:00
1953	5	1	148.02	0	7	1	148.03	0	2024-11-08	2024-11-08 15:59:00
1954	29	1	148.00	0	20	1	148.04	0	2024-11-08	2024-11-08 16:00:00

Random Number Generation

In our code we make use of an array of standard normal variables. We use the Box-Muller Transform combined with the Linear Congruential Generator algorithm to generate these variables. The Linear Congruential Generator (LCG) is

a simple algorithm for generating a sequence of pseudo-random numbers. It is defined by the recurrence relation:

$$X_{n+1} = (a \cdot X_n + c) \bmod m$$

X_n is the current value.

a is the multiplier.

c is the increment.

m is the modulus.

X_{n+1} / m maps the generated numbers to the [0,1] interval, resulting in uniform random numbers.

Once we have generated the uniform random variables from the LCG Algorithm we use the Box-Muller Transform to convert them to standard normal variables.

$$Z_1 = \sqrt{-2ln(U_1)} \cdot cos(2\pi U_2)$$

$$Z_2 = \sqrt{-2ln(U_1)} \cdot sin(2\pi U_2)$$

The Box-Muller transform works by mapping two independent uniform random variables onto the two-dimensional plane in polar coordinates and reshaping them to follow the standard normal distribution. Now, Z_1 and Z_2 are independent and follow a standard normal distribution that we can use in our simulations.

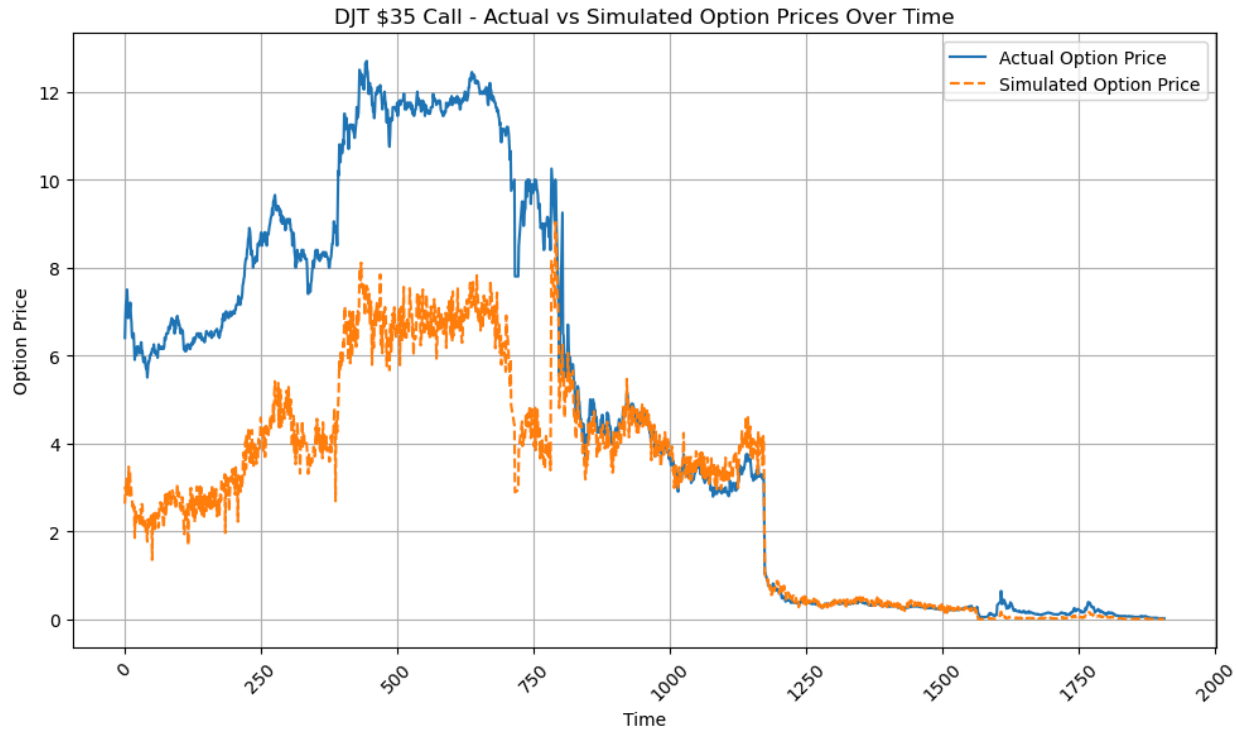
Methodology

We first use the LCG method to generate random numbers. We use these values to simulate N paths according to formula [1]. This iteratively calculates the prices over each step. Next the terminal payoff $V_T(i)$ is calculated using formula [2]. With the terminal payoff we work backwards from maturity, using polynomial regression at each time step t to estimate the coefficient of the continuation value $C_t(i)$ as in formulas [3] and [5]. Next we compare the value of exercising the option immediately to holding it for the duration as in formula [4]. Through this method we move back to time t=0 and calculate the value of the option as the mean of discounted cash flows using formula [6]. This give us a reliable estimation of the price of the options that we see tends to follow the market price very closely for some options.

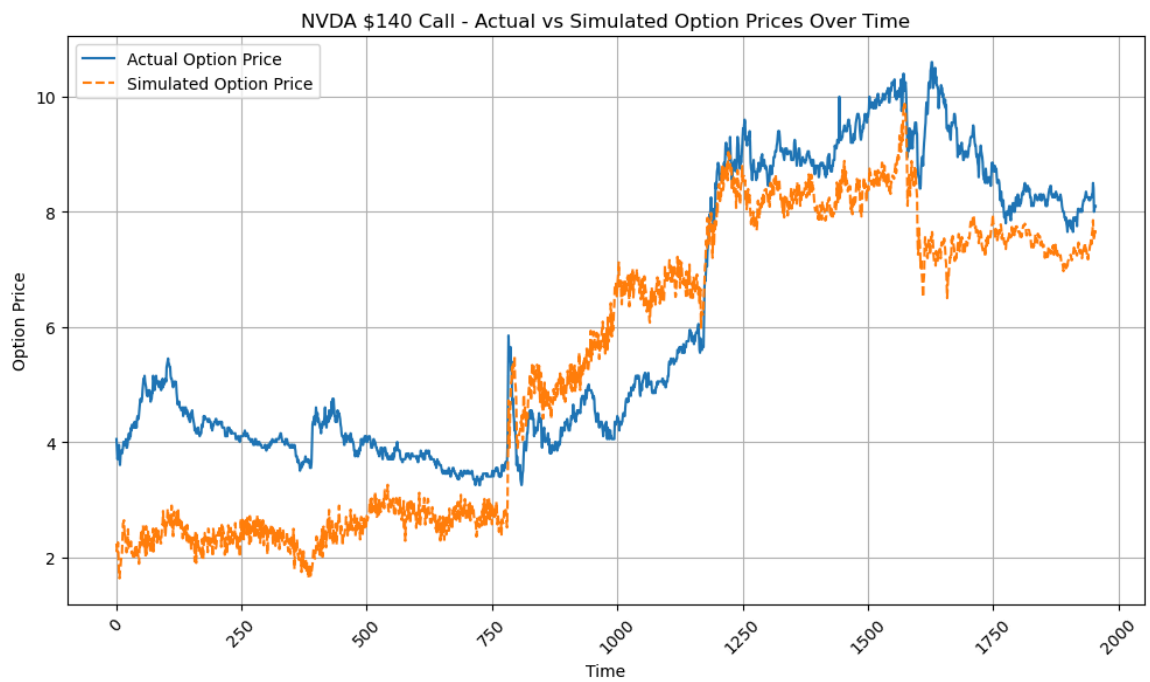
Results

Below is our simulated option price versus that market option price for calls on DJT, NVDA, AMD, and F. All comparisons take place from Nov 4th 9:30am, 2024 to Nov 8th 4pm 2024.

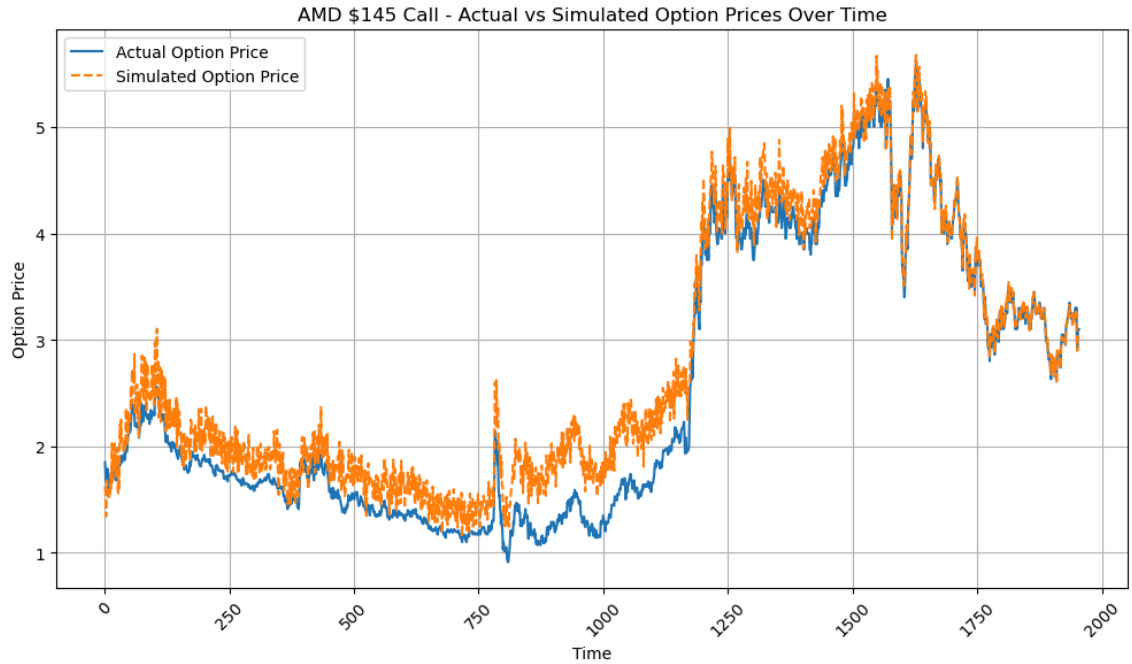
DJT $35 call: Sigma = 2.5, mean absolute error: 1.96



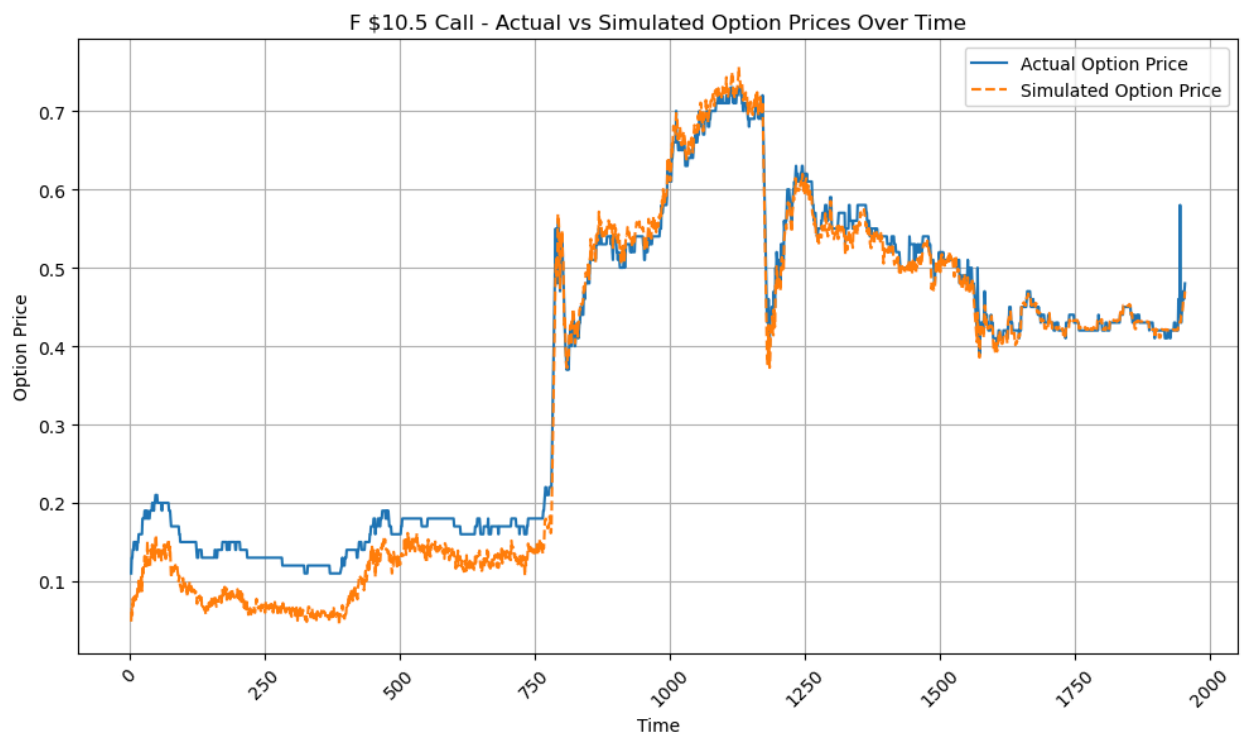
NVDA $140 call: Sigma = 0.45, mean absolute error: 1.25



AMD $145 call: Sigma = 0.45, mean absolute error: 0.24



F $10.5 call: Sigma = 0.2, mean absolute error: 0.027



We observe a lower Mean Absolute error in $10.5 call compared to AMD and NVDA's 145 and 140 calls, respectively.

Discussions

The results from our Monte Carlo simulation for pricing American options provide insight into how various factors, such as volatility, affect the accuracy of the option pricing. The results suggest that the Monte Carlo simulation struggles to price options accurately during periods of higher volatility. For example, due to increased volatility, DJT and NVDA simulated call option prices show significant deviations from the actual option price. Despite having the same volatility, the contrast between AMD and NVDA have significantly different mean absolute errors. F has the lowest volatility and the lowest mean absolute error of 0.027. This shows that Monte Carlo simulation can provide accurate pricing with minimal error on less volatile stocks. The constant volatility assumption is the limitation of Monte Carlo simulation for American option pricing. In periods of high volatility, the market often sees rapid changes in both the underlying and implied volatility, which the constant volatility fails to capture. This limitation is especially problematic for American options, where the value is path-dependent due to the possibility of early exercise. During volatile periods, frequent price fluctuations create additional potential exercise points that the model may not accurately capture with the fixed volatility. Also, while increasing the number of paths could improve accuracy, it would significantly increase the computational strain. When the analysis takes too long, the results become outdated. Overall, Monte-Carlo simulation can produce very accurate simulations for option pricing but suffers during periods of high volatility.

References-

1) Huo, J. (2023). Finite Difference Solution Ansatz Approach in Least-Squares Monte Carlo. arXiv. Retrieved from https://arxiv.org/abs/2305.09166

2) Zhu, X., Reesor, R. M., & Stentoft, L. (2024). A Critical Analysis of the Weighted Least Squares Monte Carlo Method for Pricing American Options. SSRN. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4726328

3) Djagba, P., & Ndizihiwe, C. (2024). Pricing American Options using Machine Learning Algorithms. arXiv. Retrieved from https://arxiv.org/abs/2409.03204

4) Lind, P. P., & Poulsen, R. (2024). Delta Least Squares Monte Carlo Pricing of American Options. SSRN. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4788612

5) Glau, K., Mahlstedt, M., & Pötz, C. (2018). A New Approach for American Option Pricing: The Dynamic Chebyshev Method. arXiv. Retrieved from https://arxiv.org/abs/1806.05579

6) Goudenège, L., Molent, A., & Zanette, A. (2019). Variance Reduction Applied to Machine Learning for Pricing Bermudan/American Options in High Dimension. *arXiv*. Retrieved from https://arxiv.org/abs/1903.11275

Code

```python
def lcg(seed, a, c, m, n):
    nums = []
    x = seed
    for i in range(n):
        x = (a*x+c)%m
        nums.append(x/m)
    return np.array(nums)

def generate_standard_normal(n, seed=1):
    a = 167890987
    c = 1235236237
    m = 2**16

    uniform_randoms = lcg(seed, a, c, m, 2 * n)
    u1 = uniform_randoms[::2]
    u2 = uniform_randoms[1::2]

    z1 = np.sqrt(-2*np.log(u1)) * np.cos(2*np.pi * u2)
    z2 = np.sqrt(-2*np.log(u1)) * np.sin(2*np.pi * u2)

    normals = np.concatenate((z1, z2))[:n]

    return normals
```

```python
class AmericanOptionMonteCarlo:
    def __init__(self, S0, K, T, r, sigma, M, N, option_type="put"):
        self.S0 = S0
        self.K = K
        self.T = T
        self.r = r
        self.sigma = sigma
        self.M = M
        self.N = N
        self.option_type = option_type.lower()

    def simulate_paths(self):
        dt = self.T / self.M
        S = np.zeros((self.N, self.M + 1))
        S[:, 0] = self.S0
        for t in range(1, self.M + 1):
            Z = generate_standard_normal(self.N)
            S[:, t] = S[:, t-1] * np.exp((self.r - 0.5 * self.sigma ** 2) * dt +
                                         self.sigma * np.sqrt(dt) * Z)
        return S

    def payoff(self, S):
        return np.maximum(self.K - S, 0) if self.option_type == "put" else np.maximum(S - self.K, 0)

    def price(self):
        S = self.simulate_paths()
        payoffs = self.payoff(S)
        cash_flows = np.zeros_like(payoffs)
        cash_flows[:, -1] = payoffs[:, -1]

        for t in range(self.M - 1, 0, -1):
            in_the_money = payoffs[:, t] > 0
            if np.any(in_the_money):
                S_in_the_money = S[in_the_money, t]
                X = np.vstack([np.ones(S_in_the_money.shape), S_in_the_money, S_in_the_money ** 2]).T
                Y = np.exp(-self.r * (self.T / self.M)) * cash_flows[in_the_money, t + 1]
                beta = np.linalg.lstsq(X, Y, rcond=None)[0]
                continuation_values = X @ beta
                exercise = payoffs[in_the_money, t] > continuation_values
                cash_flows[in_the_money, t] = np.where(exercise, payoffs[in_the_money, t],
                                                       np.exp(-self.r * (self.T / self.M)) * cash_flows[in_the_money, t + 1])
            cash_flows[~in_the_money, t] = np.exp(-self.r * (self.T / self.M)) * cash_flows[~in_the_money, t + 1]

        option_price = np.mean(cash_flows[:, 1])
        return option_price
```

```python
def main():
    if len(sys.argv) != 5:
        print("Usage: python script.py <path_to_df_csv> <path_to_df_stock_csv> <Strike> <Sigma>")
        sys.exit(1)

    df_path = sys.argv[1]
    df_stock_path = sys.argv[2]

    df = pd.read_csv(df_path)
    df_stock = pd.read_csv(df_stock_path)


    df['time'] = pd.to_datetime(df['time'])
    df_stock['time'] = pd.to_datetime(df_stock['time'])


    K = float(sys.argv[3])
    sigma = float(sys.argv[4])
    print(type(sigma))
    comparison_df = run_comp(K, sigma, df, df_stock)
    print(comparison_df)

if __name__ == "__main__":
    main()
```